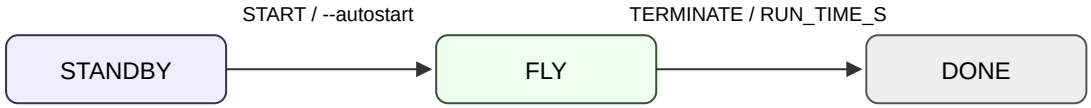
airplane: how it works

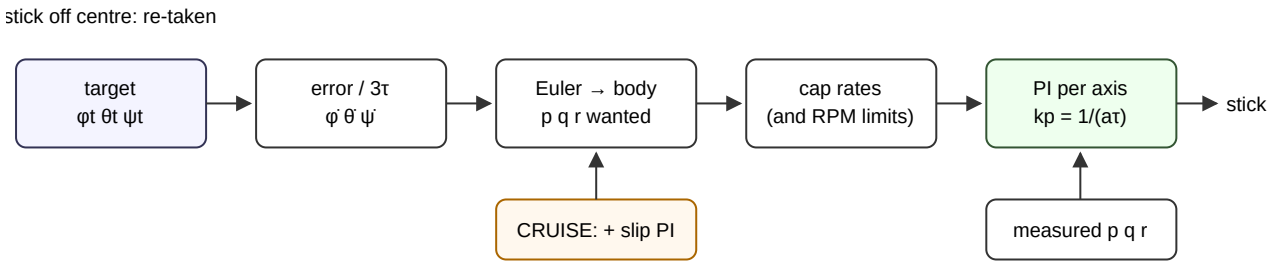
An attitude autopilot for winged craft that the player flies through. It holds the attitude on the pitch, roll and yaw inputs, and the player's stick goes on top of it. Two modes, chosen on the panel and switchable in flight: **CRUISE** (sideslip nulled, bank held, pitch attitude held through coordinated turns) and **HOLD** (pitch, heading and bank all held). **Constant rates** makes the stick command body rates, capped at limits the player sets in rev/min. **Prop plane** runs propPitch beside it.

Phases and panel



Entering FLY turns SAS off (`SAS_OFF`; restored at the end), zeroes our pitch/roll/yaw inputs and creates one PI loop per axis. The attitude at that tick is the first target. The panel (krpcLab's window) carries a CRUISE/HOLD choice, the *prop plane* and *constant rates* boxes, and the three rate limits in RPM (`PITCH_RATE_RPM` 3, `ROLL_RATE_RPM` 10, `YAW_RATE_RPM` 1.5 at start; a field that does not parse keeps the config's). A mode change re-takes the target from the attitude flown. TERMINATE sets our inputs back to 0 and SAS as it was, and stops propPitch.

Each tick



Attitude error becomes Euler rates, Euler rates become body rates, and each body rate is flown by a PI loop on its stick input whose gain comes from that axis's authority a (kRPC torque / inertia).

- **Read** (one batch): the vessel's nose, right and belly axes and its angular velocity in the vessel's surface frame (x up, y north, z east), the air velocity in the vessel's axes, speed, dynamic pressure, altitude, and the pitch/roll/yaw inputs and trims.
- **Attitude** (`control.attitude`): pitch θ = asin(nose·up), heading ψ = atan2(nose east, nose north), bank φ = atan2(−right·up, roof·up): positive right wing down, nose up, clockwise.
- **Body rates** (`control.body_rates`): read off the axes' own motion, d(axis)/dt = ω × axis: q = (ω×nose)·roof (nose going up), p = (ω×right)·belly (right wing going down), r = (ω×nose)·right (nose going right). Inputs have the same senses: pitch +1 nose up, roll +1 right, yaw +1 nose right.
- **Sideslip** (`control.sideslip`): β = atan2(v_{right}, v_{nose}) of the air velocity, positive with the air from the right, which a yaw right takes out.

The outer loops

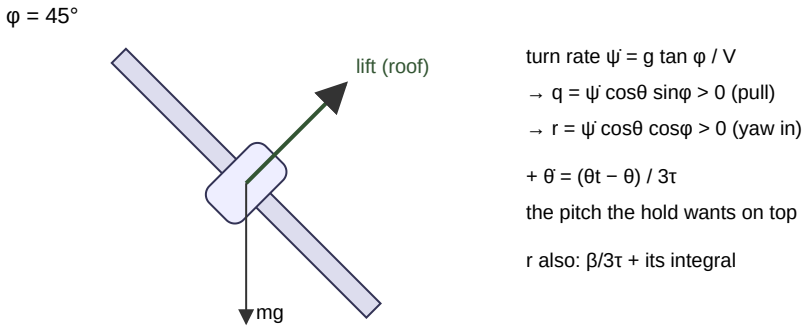
Euler rates from the attitude error, closing `ATTITUDE_TAU_FACTOR` (3) times slower than the rate loops: $\dot{\varphi} = \text{wrap}(\varphi_t - \varphi) / (3\tau)$, $\dot{\theta} = (\theta_t - \theta) / (3\tau)$, with τ the rate loops' closing time (below); the heading rate depends on the mode. They become body rates by the Euler kinematics (`control.euler_to_body`):

$$p = \dot{\varphi} - \dot{\psi} \sin\theta$$
$$q = \dot{\theta} \cos\varphi + \dot{\psi} \cos\theta \sin\varphi$$

$$r = -\dot{\theta} \sin \varphi + \dot{\psi} \cos \theta \cos \varphi$$

Each is capped at `MAX_ROLL_RATE_DEG_S` 60, `MAX_PITCH_RATE_DEG_S` 15, `MAX_YAW_RATE_DEG_S` 10, and with constant rates on also at the panel's limits.

CRUISE



Banked, CRUISE asks for the coordinated turn rate, which the kinematics turn into a pull and a yaw into the turn; so at 30° up and 45° of bank it pulls exactly as hard as keeps 30°.

- Heading is free: ψ̇ = g tan φ / V (`control.turn_rate`), with tan φ held to √(n²−1) at n = `TURN_MAX_G` (4) and V to at least `TURN_MIN_SPEED_M_S` (30). The bank used is the vessel's, held to the cap.
- The bank target is held to ±`BANK_LIMIT_DEG` (89).
- The rudder nulls sideslip: r += β/(3τ) + an integral of β/(3τ·4τ) (held to `SLIP_I_MAX_DEG_S`), unless the player is on the yaw axis.
- Past `VERTICAL_DEG` (80) of pitch the turn's feed-forward is off.

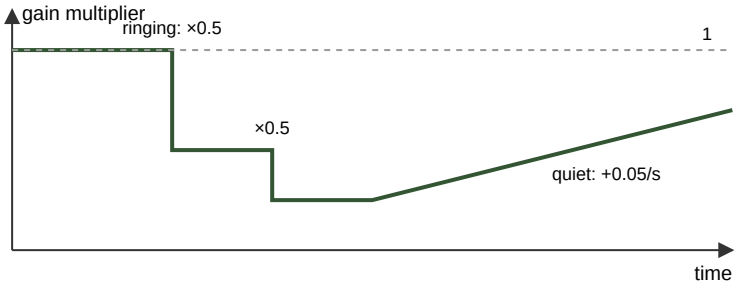
HOLD

- ψ̇ = wrap(ψt − ψ) / (3τ): pitch, heading and bank all held. No sideslip loop; no bank cap.
- Past `VERTICAL_DEG` the heading target is re-taken every tick (heading is undefined there).

The inner loops: gains from the craft

No gain is fitted to a craft. Per axis (pitch flies q, roll p, yaw r):

- **Authority** a (`control.authority`): kRPC's `available_torque` — reaction wheels, RCS, engine gimbals and the control surfaces at the present dynamic pressure and Mach — the mean of the two directions, over `moment_of_inertia` on that axis: the angular acceleration a full input gives. Re-read every `AUTHORITY_REFRESH_S` (0.5 s game time); logged at START.
- **Closing time** τ, per axis: at least `RATE_TAU_TICKS` (4) × the loop's own measured interval (game seconds between ticks, smoothed) and `RATE_TAU_MIN_S` (0.15 s) — the loop is as fast as it can serve — and no faster than the axis's surfaces can move (next section).
- **Gains** (`control.rate_gains`): kp = 1/(aτ), the input that takes the rate error out in τ; ki = kp/(`INTEGRAL_TAU_FACTOR`·τ) (4); with no authority, `KP_MAX` (20).
- **Loop** (`control.RateLoop`): u = kp·e + I, e the wanted rate less the measured one (rad/s), I += ki·e·dt held to ±`INTEGRAL_MAX` (0.6) and not wound further while u is pinned at ±1 that way, u clamped to ±1. The integral is the axis's trim. An input is written when it moved by 0.001.
- **Ringing** (`control.Ringing`): kRPC models the surfaces rather than measuring them. Where its figure is high the loop rings: when an axis's output has reversed `RING_REVERSALS` (4) times by `RING_SWING` (0.15) within `RING_WINDOW_S` (2 s), its kp and ki are halved (at most once per window, never below `RING_FLOOR`, 1/8); with no reversal in the window the multiplier climbs back toward 1 by `RING_RECOVER_PER_S` (0.05/s). An axis the player flies by hand is not counted.

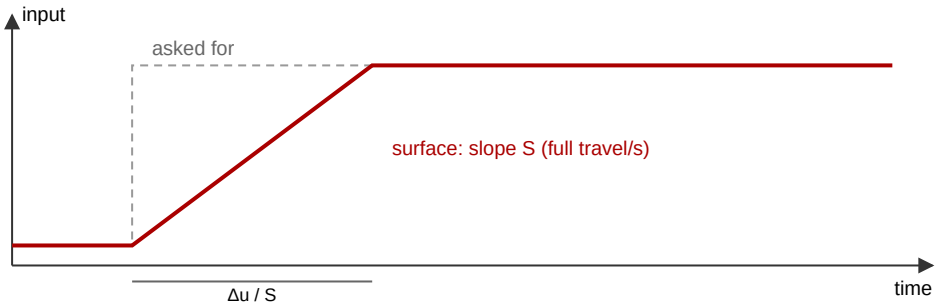


An axis that rings has its gain halved; quiet, it climbs back toward the authority's own gain.

The surfaces' travel time

KSP moves a control surface's input toward the command by `actuatorSpeed`·dt/`ctrlSurfaceRange` of full travel per physics step, and its angle at `actuatorSpeed` deg/s (`ModuleControlSurface`; with `useExponentialSpeed` both are first-order lags instead). The window mod's `SurfaceActuators` reads, every `AUTHORITY_REFRESH_S`, the vessel's summed potential torque and each control surface's speed, range, mode and potential torque. Per axis (`control.actuators`):

- **Slew** S (full travel per second): each linear surface's `actuatorSpeed`/`ctrlSurfaceRange`, combined as total torque / Σ(torque/S); torque not on a slewed surface (wheels, RCS, gimbals) counts as instant. None when no slewed surface acts.
- **Lag** L: exponential surfaces' range/speed + 1/speed, weighted by their share of the torque.



A surface reaches a new input Δu only after Δu/S; a loop that expects it sooner overshoots.

Then (`control.closing_time`), each axis's closing time τ is the largest of: the tick bound above; the time the surfaces need to deliver a rate step of the axis's cap c — the input it needs is c/(aτ) and it has to get there within τ/`SLEW_FACTOR` (2), so τ ≥ √(2c/(aS)); and `LAG_FACTOR` (3) × L. The gains and the attitude loops of that axis follow from its τ. And `RateLoop` moves its command at most S·dt a tick — never faster than the surfaces — with the integral not winding against that limit. Engine gimbals' response is not modelled. Without the window mod every axis is taken as instant.

The player's stick

kRPC adds a client's input to the player's (`PilotAddon.OnFlyByWire` adds its `manualInputs` to the game's state) and reads back the sum; KSP has already folded the trim into the stick (`FlightInputHandler`). So the player's own input on an axis is the read-back less what we wrote last tick less the trim (`control.stick`). Off centre by more than `STICK_DEADBAND` (0.05):

- **Constant rates off**: the player flies that axis. Our input there is only the loop's integral (no proportional part, the integral frozen), and the target on that axis follows the attitude flown (pitch stick → θt, roll → φt, yaw → ψt). When the stick returns inside the deadband, the attitude it left is held. The other axes keep holding: in CRUISE a rolled-in bank is pulled and coordinated as above.
- **Constant rates on**: the stick is a rate command, wanted = stick × limit (RPM × 2π/60 rad/s), flown by the loop; since the game adds the stick to our input, ours is the loop's output less the stick. A full stick at

high speed therefore turns at the limit and no faster; at low speed the loop runs out of travel first. The target follows as above.

Prop plane

With the box ticked, a `propPitch.PropPilot` runs in the same process, ticked after the airplane each tick, with its own LOG and a panel that draws nothing; its blade angle, rpm and mode show on the airplane's panel. It starts at once (autostart). Unticking it, or TERMINATE, shuts it down (rotors and KALs handed back). If it cannot start (no rotor with blades) or fails, the error shows on the panel and it is not retried until the box is ticked again.

The log

`FLY CRUISE|HOLD`	the mode; `const` at the end with constant rates on
`spd= alt= q=`	surface speed, altitude, dynamic pressure (Pa)
`att=` / `tgt=`	bank, pitch, heading flown / held (deg)
`slip=`	sideslip (deg)
`rate=` / `want=`	pitch, roll, yaw body rates measured / wanted (deg/s)
`out=` / `i=`	our pitch, roll, yaw inputs / their integrals
`stick=`	the player's input per axis
`kp=` / `acc=`	the rate loops' proportional gains / the authority they came from (deg/s² per unit input)
`ring=` / `tau=`	each axis's ringing multiplier / closing time (s), pitch, roll, yaw
events	`authority …`, `surface actuators …` (START), `mode …`, `stick <axis> flying/released`, `<axis> ringing: gain ×k`, `prop plane: …`